

Aircall - Interview Prep

What do they do?

Shot answer:

Aircall is a cloud phone system that captures every call as data, ties it to the right contact, pushes it into the tools a team already uses (CRM, Slack, ticketing), and layers AI on top to summarize, structure, and even handle calls. The phone number is the key, the CRM is the record, Aircall is the engine in between.

Long answer:

Aircall is a cloud phone system for customer-facing teams. It brings voice, SMS, and WhatsApp into one place, and it connects to the tools companies already use, like Salesforce, HubSpot, Zendesk, or Slack. So every call gets captured and tied to the right contact, instead of call data living in one system and the CRM in another. On top of that there's an AI layer: a Voice Agent that handles routine calls, and tools that summarize and structure the calls a person takes. They work with over 22,000 companies, from small teams to enterprise, and Europe is a big part of that - Paris is the HQ and Madrid is a major hub. What caught my eye is that it's the same bridge I've been doing for the last few years: the technical integration on one side, the customer on the other, plus this growing data and AI piece that I find genuinely interesting.

There's also a reporting and conversation-intelligence layer, so teams can see call volumes, handle time, and what actually happened on each call. That part maps to work I've done too - I built data-visualization dashboards earlier in my career and later a data-extraction product at N-SIDE.



Tell me about you

I'm a **Solutions Engineer**. For the last five years at N-SIDE I've worked 50/50 as a Solutions Engineer and a Developer on an API platform. On the Solutions Engineer side I **owned the customer relationship**: I designed around **20 API endpoints**, took the product from a **PoC to production**, onboarded five **customers**, wrote all the external **documentation** and tutorials, and handled their technical problems directly. On the developer side I built the platform itself, on the TYK gateway. So I sit exactly where Aircall puts a Customer Engineer: a Solutions Engineer sitting between the product and engineering side and the customer.

How was your SE daily work?

Customers submitted issues through a ticket portal. A first line triaged them; I handled the escalated technical and integration issues as the second line. I also wrote the API documentation and tutorials that reduced the volume reaching us.

And because I was hearing the problems first-hand, I became the point of contact for the product managers: identified pain points and possible improvements, assessed and measured new ideas, and turned them into features; worked between customers and Product/Engineering;

Why Aircall?

The discovery, the integrations, seeing someone actually adopt what we built. That's what motivates me most right now. For me it's the chance to go from 50/50 to full Solutions Engineering - that's what I want right now. Aircall's Customer Engineer is the full version of the customer-facing work I've been doing half-time and I can still leverage my technical knowledge with APIs.

The level question: Is staff engineer too high?

One thing I wanted to align on: I applied to the Customer Engineer role, and your email mentioned the Staff position. I'd love to understand whether this is the same opening leveled up, or a separate req. I'm flexible on level - my priority is the scope and the team. I have 10+ years in engineering, the last ~5 at N-SIDE building and supporting an API product directly with customers, so I'd like to understand where that lands on your ladder

Indeed, this isn't a good fit for you

Understood. Would you also consider me for the Customer Engineer opening? It's still live and I think it's a strong fit.

Are you comfortable with Staff level?

Be honest: "I'm confident on the technical and customer sides. The newer part for me is the enterprise/commercial weight - exec QBRs, renewal and NRR ownership at scale. I'm keen to grow into that, and I pick up domains fast."

STAR stories

The API branch

Ownership / PoC to production · solution architecture & technical depth · driving adoption · commercial / expansion instinct.

- **Situation:** customers did everything by hand in the UI - uploading data, running simulations, copying results out; a key client asked to connect their own systems instead.
- **Task:** take it on as a PoC but treat it like a product.
- **Action:** discovery with the customer and their technical users (what they needed to push in and pull out); designed ~20 API endpoints; built the platform on the TYK gateway; wrote the external docs and video tutorials; handled their technical questions directly and iterated on what broke.
- **Result:** full product adopted by every existing account; 5 new customers integrated after launch; a new revenue line from the same API (customers pulling their own historical data - shipments, CO2 emissions, trial duration).

Support flow and product feedback

Troubleshooting complex issues · cross-functional influence · honest handling of a missing feature · calm under pressure.

- **Situation:** customers submitted issues through a ticket portal; a first line triaged them; I was the second line for the technical and integration problems.
- **Task:** own the escalated problems end to end - and turn the recurring ones into product input instead of fixing the same thing forever.
- **Action:** debugged customer integrations against the API (what changed, logs, auth, webhooks); wrote the docs, tutorials, and videos that cut the volume reaching us; became the point of contact for PMs - identified pain points and possible improvements, assessed and measured new ideas, and turned them into features; when a request wasn't on the roadmap, found a workaround and logged the business case.
- **Result:** fewer support tickets; recurring issues became product improvements; trust kept when I couldn't hand over the feature.

Performance incident on one instance

Troubleshooting under pressure · owning an incident end to end · prevention and lessons learned.

- **Situation:** one customer's instance became unresponsive. A single endpoint was doing heavy computation, and a specific corner case - an unusual combination of shipments, locations, patients, and other parameters - exploded the computation time. It froze that instance, so even normal users lost the UI.
- **Task:** restore the instance, find the root cause, and make sure it couldn't happen again.
- **Action:** we located the endpoint from monitoring and logs, reproduced it with the customer's data set, and profiled the computation. We mitigated first - protecting the instance by limiting or queueing the heavy operation - then fixed the root cause by optimizing the calculation.
- **Result:** the instance recovered, normal users were unblocked, and the endpoint's response time came back in line.

- **Prevention:** we added safeguards so it couldn't recur - per-endpoint protection (timeouts and request limits, moving heavy work to an async job), monitoring and alerting on response time, and a regression test for that exact corner case. We treated it as a prevention item, not a one-off fix

Gateway vs BFF (the architecture lesson)

solution-design judgment · owning a mistake honestly · putting logic in the right layer.

- **Situation:** our clients wanted to connect their systems to our SaaS and get clean, filtered data, but the internal backend only exposed big, generic chunks - it wasn't built for that.
- **Task:** give them a proper API quickly, without waiting on the slow core release cycle.
- **Action:** the Gateway was already sitting in front, so I used its built-in scripting to filter and reshape the responses there. For the pilot it worked, and it looked like we'd avoided building a service.
- **Result / what went wrong:** as more clients asked for different filters, merges, and shapes, that logic grew inside the gateway. It was hard to test, every change meant a gateway deploy, and we mixed "edge" concerns (keys, rate limits, routing) with "application" concerns (filtering, merging). We'd built a BFF in the wrong place.
- **Close / lesson:** the right design is a gateway at the edge and a small application layer - a BFF (backend-for-frontend) - behind it: the gateway does auth, keys, rate limits, and routing; the BFF does the filtering, merging, and reshaping, and gets tested and deployed like normal code.

In the end that's exactly what we did - we implemented the BFF properly as its own layer and kept the gateway for what it's good at. The lesson I carry: put logic in the right layer, not the convenient one.

That gives us the flexibility to experiment and the robustness to productify mature endpoints.

Native endpoint for a repeated BFF pattern

Spotting patterns · proposing product improvements · cross-team collaboration · performance.

- **Situation:** the BFF (backend-for-frontend) was making a repeated call pattern - several calls to the internal API product just to build one screen or action.
- **Task:** cut the load and the latency without changing anything the user sees.
- **Action:** I looked at the call pattern, confirmed it was frequent and costly, and proposed a native endpoint in the internal API that returned the needed data in a single call. Worked with the team to design and ship it, then moved the BFF onto it.
- **Result:** the BFF got faster and made far fewer calls, and the load on the internal API dropped. Other consumers could reuse the same endpoint.
- **Close:** "spot the pattern, propose the product change, ship it, measure it" - the same motion I'd run with customers on Aircall.

-

● Gap awareness

Business domain

Problem: The biggest gap is the domain. I've spent my career in APIs, integrations, and platforms - not telephony, not living inside Salesforce or HubSpot, and not voice AI. SIP, call flows, contact-center metrics, CRM depth, agentic voice - that's all product knowledge I'd be picking up.

Action: I'd want a structured ramp: your product bootcamp, shadowing a senior CE through two or three onboardings, and pairing on live escalations. Domain knowledge is the fastest thing for me to learn - I've done exactly that before.

Strategic and executive stakeholders

Problem: Most of my customers were mid-market and technical users. I haven't spent much time in front of CTOs or Heads of CX.

Action: I'd start by joining a senior CE on a couple of enterprise accounts and one QBR, then take my own from there. Seeing it run once is the fastest way for me to pick it up.

French level

Problem: French is my third language, and I use it every day - my current company is based in Belgium, so a good share of internal meetings and discussions happen in French, and I'm comfortable thinking and speaking in it. Where I'm lighter is customer-facing French, because my customer work has mostly been in English.

So it's about sharpening the customer vocabulary and register, not about learning the language."

Action: I'd sit in on French customer conversations with a colleague for my first few weeks and practise the product vocabulary. I'm confident I'd be fully operational in a French or hybrid environment after a short ramp - I just want to be straight that the customer-facing side would take me a little time to warm up, since it's a different register from internal meetings."

Mentoring at Staff scale

Problem: I've led a Kanban team, onboarded peers and customers and written technical documentation, but I haven't mentored a team of CEs

Action: I'd start the same way: find out from the CEs where they get stuck, then fix the docs and run sessions on those exact things.

Common questions:

- What are you looking for in your next role?
- Where do you want to be in a few years?
- Salary expectations? 80k/90k
- Location / relocation to Madrid / hybrid? Based in Belgium, Spanish native, happy to relocate (confirm your actual situation).